

CORTEX: A Unified Cognitive Tree Architecture for Persistent, Navigable, and Self-Consolidating LLM Agent Memory

Author One¹ Author Two¹ Author Three¹

¹*Affiliation, Address*

Correspondence: author1@org.tld

Preprint — May 2026

Abstract

Long-running LLM agents face a structural tension: their reasoning is bounded by a finite context window, yet useful tasks demand continuity across hours, days, and many tool invocations. Existing agent memory systems address parts of the problem — episodic logs (Mem0, Zep), tiered context management (MemGPT/Letta), reflection trees (Generative Agents), reasoning-based retrieval (PageIndex), and recursive task decomposition (RLM) — but no public framework integrates a writable cognitive workspace, persistent multi-domain memory, automatic consolidation, sub-agent isolation, and an associative graph overlay into a single coherent runtime. We present CORTEX (Cognitive Orchestrated Recursive Tree EXecution), an agent-memory architecture in which the agent’s complete cognitive state is a typed, persistent tree, and its language-model context window is merely a bounded *viewport* onto that tree. CORTEX unifies four memory domains — Knowledge, Episodic, Experience, and Intelligence — onto a single ORM model trio (**tree** / **node** / **edge**), disambiguated by enumeration. A background “Dreaming” pipeline consolidates episodes into observations, observations into patterns, and patterns into actionable rules, mirroring established cognitive-science models. A semantic graph layer overlaying all trees enables hybrid embedding-plus-graph retrieval with auto-decay maintenance. We describe the data model, the seven primitive operations, the consolidation pipeline, the multi-tenant scope hierarchy, integration with an execution engine, and the system’s invariants. We characterize CORTEX’s design properties, position it against the contemporary memory-framework landscape, and discuss limitations and open questions.

Keywords: large language models, agent memory, cognitive architecture, hierarchical retrieval, knowledge consolidation, multi-agent systems, context engineering.

1 Introduction

The capabilities of large language model (LLM) agents have advanced rapidly, yet four structural problems persist across deployed systems:

1. **Context rot.** Empirical and anecdotal evidence shows that model accuracy degrades as prompt length grows, even within stated context limits — a phenomenon Anthropic terms “context rot.” Filling the window with running transcripts is therefore not a viable long-task strategy.
2. **Opaque retrieval.** Vector-based retrieval-augmented generation (RAG) flattens document hierarchy into embeddings, sacrificing structural information and provenance. Recent work (PageIndex, Vectify AI 2025) demonstrates that reasoning-based retrieval over a hierarchical document tree can lift FinanceBench accuracy from approximately 31% to 98.7%.

3. **Cross-run amnesia.** Most production agents start each invocation from a blank slate. Persistent memory frameworks like Mem0 and Zep address this for facts but do not provide a writable workspace the agent can edit and revisit.
4. **No learning loop.** Systems that record execution history typically lack a mechanism to abstract that history into reusable rules. The cognitive-science literature on *memory consolidation* — episodic experience compressed into semantic knowledge — has obvious parallels but is implemented in few production systems.

We argue that addressing these four problems requires not a new model, but a new *runtime substrate* for the agent’s cognitive state. Specifically, we propose that the agent’s working state should be modeled as a persistent, navigable, writable typed tree, and that the language model itself should only ever see a bounded slice — a *viewport* — of that tree. All other context is reachable by navigation, paging, or sub-task delegation.

Contributions. This paper makes the following contributions:

1. **A unified data model** in which four orthogonal memory domains (Knowledge, Episodic, Experience, Intelligence) and a transient runtime workspace share a single ORM trio (**tree** / **node** / **edge**), disambiguated by enumeration. One set of query, retrieval, graph, and embedding primitives serves every domain (§3, §4).
2. **Seven primitive operations** (NAVIGATE, READ, WRITE, RECURSE, AWAIT_CHILDREN, CHECKPOINT, ASSEMBLE) that an agent invokes to manipulate its cognitive tree, formalizing the “agent-as-tree-explorer” pattern. We elevate these to first-class planner step types rather than hiding them behind tool calls (§5).
3. **A Dreaming consolidation pipeline** that runs the cognitive sequence $episode \rightarrow observation \rightarrow pattern \rightarrow rule$ as a typed three-phase background job with confidence and recurrence thresholds, embedding-based clustering, and a generation counter (§6).
4. **A semantic-graph overlay** (`cortex_edges`) enabling associative cross-tree, cross-domain retrieval with auto-edge creation on embedding, weight decay over inactivity, and co-access tracking during execution (§7).
5. **A six-level scope hierarchy** ($App \rightarrow Partner \rightarrow Tenant \rightarrow User \rightarrow Entity \rightarrow Runtime$) that unifies platform-wide, multi-tenant, per-user, per-agent, and per-run state within the same schema (§8).
6. **Subtree-isolated recursive children** via a recursive-CTE ancestry check, solving the “sub-agent leaking context to parent” failure mode common in multi-agent systems (§5.4).

We position CORTEX against contemporary frameworks in §2, describe the architecture and implementation in §3–§9, characterize design properties in §10, discuss limitations in §11, and outline future work in §12.

2 Related Work

2.1 Tiered and Operating-System-Inspired Memory

MemGPT [1], now developed as Letta, treats the LLM as an operating system with core memory, archival memory, and recall memory tiers, paging information between tiers via tool calls. CORTEX adopts the paging idea but extends it from a tiered store to a typed tree, adds four orthogonal domains rather than three abstract tiers, and supplements the agent-driven paging with system-driven background consolidation.

2.2 Memory-as-a-Service Frameworks

Mem0 [2] extracts facts from conversations into a hybrid vector + graph + key-value store with user, session, and agent scopes. Zep introduces a temporal knowledge graph capturing state changes (“I used to live in London, but I moved to Tokyo”). Cognee emphasizes deep knowledge graphs. These systems excel at fact recall but do not provide a writable workspace, recursive sub-task scoping, or output canvas.

2.3 Reflection and Generative Agents

Park et al. [3] introduced reflection trees in which agents periodically summarize the latest ~100 memories into higher-level insights, retrieved via a weighted combination of recency, importance, and relevance. CORTEX’s Dreaming Engine generalizes this from a single LLM reflection prompt to a typed three-phase pipeline with explicit confidence/strength thresholds and edge creation between phases.

2.4 Cognitive-Architecture-Inspired Frameworks

The Cognitive Architectures for Language Agents framework [4] (CoALA) provides the canonical taxonomy: working, episodic, semantic, and procedural memory. CORTEX is a concrete instantiation of CoALA with one refinement: it splits CoALA’s “semantic + procedural” into three orthogonal domains (Knowledge / Experience / Intelligence), with Experience serving as the explicit consolidation midpoint between raw episodes and actionable rules. The classical cognitive architectures Soar [5] and ACT-R provide the broader inspiration — production rules, declarative memory, and chunking.

2.5 Reasoning-Based Retrieval

PageIndex [6] demonstrates that hierarchical tree navigation by an LLM agent outperforms vector search on structured documents. CORTEX adopts PageIndex’s bounded-viewport navigation primitive but extends the tree from a read-only document index to a writable workspace that persists across runs and overlays a vector + graph layer for hybrid retrieval rather than purely-reasoning retrieval.

2.6 Recursive Language Models

Recursive Language Models (RLM) [7] let the LLM treat the prompt as a variable and recursively sub-query itself, achieving robust performance at 10M+ token inputs. CORTEX’s RECURSE primitive applies the same idea to a typed tree: a child run is spawned scoped to a subtree (`scoped_subtree_root_id`) and results are collected via `AWAIT_CHILDREN`. The tree itself is the bounded environment.

2.7 Hierarchical Multi-Agent Memory

G-Memory [8] introduces a three-tier hierarchical memory (insight / query / interaction graphs) for multi-agent systems. ByteRover [9] represents knowledge in a Domain → Topic → Subtopic → Entry context tree with importance scoring and maturity tiers. CORTEX overlaps these on importance scoring (`importance_score`), generation counters (`consolidation_generation`), and decay (`access_count`, edge weight decay), while broadening scope coverage and integrating tightly with an execution engine.

2.8 Filesystem-as-Memory

Cognitive Workspace [10] and Letta’s filesystem-benchmark work argue that a plain filesystem with markdown notes can match or beat vector RAG for agents that manage their own scratchpad. CORTEX can be viewed as a *schematized* filesystem: every “file” carries type, status, embedding, parent, summary, content, provenance, importance, and edges. The structure enables both LLM navigation and SQL queries — a property a plain filesystem lacks.

2.9 Summary of Positioning

Individual ideas in CORTEX have published antecedents. The contribution is an **integrated runtime**: to our knowledge, no public framework simultaneously provides (a) a single schema unifying knowledge, episodic, experience, intelligence, and runtime working state; (b) six-level scope inheritance; (c) automatic Dreaming-style consolidation; (d) viewport-based bounded context with paged reads; (e) recursive subtree-isolated children; and (f) a graph layer with auto-edge creation, decay, and co-access tracking, behind one ORM model and seven primitive operations.

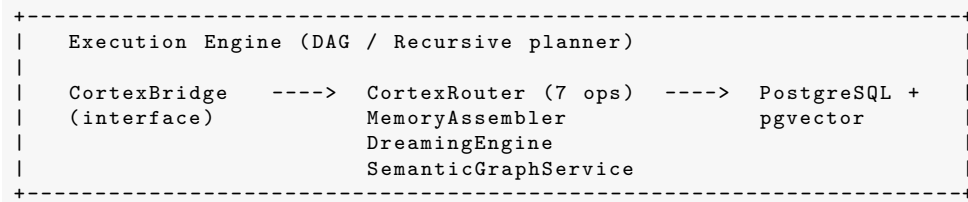
3 System Overview

3.1 Core Thesis

The agent’s complete cognitive state is the tree. The language model’s context window is a viewport onto the tree.

Every observation, finding, knowledge reference, sub-task, checkpoint, and output paragraph the agent produces is a node in a persistent typed tree. The LLM’s input at any step is the rendered viewport of the current cursor node — its title, summary, immediate children’s summaries, parent, and breadcrumb — plus an enumeration of the seven CORTEX operations the agent can invoke.

3.2 Logical Layering



3.3 The Four Memory Domains

CORTEX persists four orthogonal long-lived trees per agent entity, plus one transient runtime tree per execution (Table 1).

3.4 Design Principles

CORTEX is built on four invariants that are enforced in code:

1. **Summary Always Exists.** A node cannot have children unless its **summary** is set. This guarantees a parent’s viewport is always informative.
2. **No Unbounded Viewports.** A node’s direct-children count is bounded by **max_children** (default 12). Exceeding the bound triggers asynchronous re-clustering into a group node.

Table 1: The four persistent memory domains and the transient runtime workspace.

Domain	Source	Written by	Used for
Knowledge	User documents, scraped data	Ingestion pipelines	Reference recall
Episodic	Completed execution runs	System (on run completion)	Recent history
Experience	Distilled from episodes	Dreaming Engine (phases 1–2)	Observations and patterns
Intelligence	Distilled from patterns	Dreaming Engine (phase 3)	Actionable rules
Runtime (<i>transient</i>)	The current execution	Agent and execution engine	Working memory, current output

3. **Content is Always Paged.** Large `content` fields are read in slices of `page_size_tokens` (default 8000 tokens). No single read returns more than one page.
4. **Write-Once Content.** Node content is immutable; revisions are modeled as child nodes, not in-place edits. This makes provenance traceable and prevents subtle race conditions.

4 Data Model

CORTEX uses three tables stored in PostgreSQL with the `pgvector` extension.

4.1 The `cortex_trees` Table

Each tree row is a cognitive context. Key columns:

- `id`, `entity_id`, `user_id`, `company_id` — ownership and tenancy.
- `task_description`, `status` $\in$ {active, suspended, complete, archived}.
- `root_node_id`, `output_root_id`, `resume_cursor_id` — cursors into the node table. `resume_cursor_id` is the single source of truth for “where the agent is.”
- `max_children` (default 12), `page_size_tokens` (default 8000), `context_budget_pct` (default 40) — invariants and tunables.
- `memory_domain` $\in$ {knowledge, episodic, experience, intelligence}, `scope_level` $\in$ {app, partner, tenant, user, entity, runtime} — the two enums that disambiguate every tree.
- `last_consolidated_at`, `consolidation_generation`, `source_run_ids` — Dreaming meta-data.
- `resume_schedule`, `next_resume_at` — multi-day scheduled wake-ups.

4.2 The `cortex_nodes` Table

Every piece of information is a node. Key columns:

- `id`, `tree_id`, `parent_id` — structural.
- `node_type` — one of 18 typed values: `root`, `knowledge`, `finding`, `task`, `output`, `checkpoint`, `group`, `document`, `section`, `chunk`, `observation`, `pattern`, `suggestion`, `instruction`, `strategy`, `preference`, `episode`, `episode_group`.

- `title`, `summary`, `content`, `content_tokens` — display and content.
- `status` $\in \{\text{pending, active, complete, summarised}\}$.
- `source_ref`, `execution_run_id`, `metadata_extra` — provenance.
- `embedding` (`pgvector.Vector(768)`), `embedding_model` — semantic vector.
- `cross_refs`, `access_count`, `last_accessed_at`, `importance_score` — graph and learning signals.

4.3 The `cortex_edges` Table

Directed, weighted, typed edges overlaying all trees. Edges may cross trees, domains, and scope levels.

- `source_node_id`, `target_node_id`, `edge_type`, `weight` $\in [0, 1]$.
- `traversal_count`, `last_traversed_at` — for boosting and decay.
- `created_by` — provenance string (e.g., `"dreaming_engine"`, `"embedding_pipeline"`).
- Uniqueness: (`source`, `target`, `edge_type`). Inserts are upserts.

The edge-type vocabulary is given in Table 2.

Table 2: Edge-type vocabulary in the semantic graph layer.

Edge type	Semantics	Created by
<code>semantic_similar</code>	Cosine similarity ≥ 0.85	Auto on embedding
<code>derived_from</code>	Pattern derived from observation	Dreaming phase 2
<code>generalizes</code>	Rule generalizes a pattern	Dreaming phase 3
<code>co_accessed</code>	Read together in the same step	Runtime tracking
<code>references</code>	Citation between nodes	Ingestion
<code>precedes</code> , <code>contradicts</code> , <code>supersedes</code> , <code>applies_to</code>	Reserved for future use	—

4.4 Schema Property: One Trio Generalizes All Domains

Because the three tables are disambiguated only by enums, the same `CortexRouter.read`, `write`, `navigate`, vector-search SQL, and graph-traversal CTEs work uniformly across Knowledge, Episodic, Experience, Intelligence, and Runtime trees. New domains can be added by extending the enum without altering the storage layer.

5 The CORTEX Operations

The `CortexRouter` class implements seven primitive operations. These are surfaced to the agent both as planner step types (`NAVIGATE`, `READ`, `WRITE`, `RECURSE`, `AWAIT_CHILDREN`) and as direct method calls (`CHECKPOINT`, `ASSEMBLE`).

5.1 The Seven Primitives

Table 3 lists the operations.

Table 3: The seven primitive CORTEX operations.

Op	Signature	Behavior
NAVIGATE	<code>navigate(node_id) → Viewport</code>	Move cursor; return current + children + parent + breadcrumb
READ	<code>read(node_id, page=0) → NodeContent</code>	Return one page; promote node pending → active
WRITE	<code>write(parent_id, type, title, content, summary, ...) → UUID</code>	Create child node; enforce invariants 1 and 2
RECURSE	<code>recurse(node_id, task, result_slot, ...) → (task_node_id, child_run_id)</code>	Create scoped-subtree child execution
AWAIT_CHILDREN	<code>await_children(parent_id) → {slot: NodeSummary}</code>	Collect results from completed child task nodes
CHECKPOINT	<code>checkpoint(tree_id, progress, key_facts, next_steps) → UUID</code>	Compress context into a checkpoint node
ASSEMBLE	<code>assemble_output(tree_id, coherence_pass=True) → str</code>	DFS the Output subtree; LLM-generate transition paragraphs

5.2 The Viewport

A viewport is the bounded representation of the agent’s current position. Token cost is bounded by $\text{max_children} \times \sim 40$ tokens per child summary, plus the current node’s summary, the parent’s summary, and the breadcrumb. The viewport is serialized as structured text:

```
## Navigation Path
<breadcrumb>

## Current Node: <title>
Type / Status / Depth / Summary

## Children
[1] <title> (<type>, <status>) -- <summary>
...

## Available CORTEX Operations
NAVIGATE(...) | READ(...) | WRITE(...) | RECURSE(...) |
AWAIT_CHILDREN() | CHECKPOINT(...)
```

5.3 Bounded Context Property

Property 1 (*Bounded Viewport*). For any tree T with $\text{max_children} = M$, the prompt cost of $\text{navigate}(n)$ for any node $n \in T$ is $O(M + \text{depth}(n))$ tokens, independent of $|T|$.

Justification. The viewport contains the current node’s summary ($O(1)$), at most M child summaries ($O(M)$), one parent summary ($O(1)$), and a breadcrumb whose length equals $\text{depth}(n)$. $\square$

This property is the central design invariant: the agent’s prompt cost is decoupled from the tree’s size. A tree with 100,000 nodes presents the same per-step prompt cost as a tree with 10 nodes.

5.4 Subtree Isolation

When a `CortexRouter` is constructed with `scoped_subtree_root_id`, every `_get_node` call validates that the requested node is a descendant of the scope root via a single recursive CTE:

```
WITH RECURSIVE ancestors AS (  
  SELECT id, parent_id FROM cortex_nodes WHERE id = :node_id  
  UNION ALL  
  SELECT cn.id, cn.parent_id  
  FROM cortex_nodes cn JOIN ancestors a ON cn.id = a.parent_id  
)  
SELECT 1 FROM ancestors WHERE id = :ancestor_id LIMIT 1;
```

Out-of-scope access raises an error.

Property 2 (*Subtree Isolation*). A child execution constructed with `scoped_subtree_root_id = s` cannot read or write any node n such that s is not an ancestor of n .

5.5 Re-Clustering on Viewport Overflow

When `WRITE` would cause a node's child count to exceed `max_children`, an asynchronous re-clustering routine moves the first half of children under a new `group` node. The group inherits the type of the moved children, preserving semantic homogeneity within groups.

5.6 Resumability

A tree maintains `resume_cursor_id` updated by every `NAVIGATE`, `READ`, and `WRITE`. On `resume_tree(tree_id)`:

1. Status is flipped from `suspended` to `active`.
2. The viewport at `resume_cursor_id` is loaded.
3. The most recent `checkpoint` node under the cursor is returned.
4. The execution engine's `__completed_steps__` set is restored, causing already-finished steps to be skipped.

This makes resumption deterministic and idempotent at step granularity.

6 The Dreaming Consolidation Pipeline

The `DreamingEngine` runs the cognitive sequence $episode \rightarrow observation \rightarrow pattern \rightarrow rule$ as a three-phase background job.

6.1 Trigger

The pipeline runs when:

- `dreaming_worker` is enqueued (post-run hook or scheduled), and
- `now() - experience_tree.last_consolidated_at ≥ CONSOLIDATION_INTERVAL_HOURS` (default 24), or
- `force=True`.

6.2 Phase 1 — Observation Extraction

Inputs: episodes since `last_consolidated_at` (capped at `BATCH_SIZE = 20`), minimum `MIN_EPISODES_FOR_DREAMING = 5`.

The engine builds episode summaries `{id, task, status, tools_used, cost_usd, execution_time_ms}` and calls the LLM with `OBSERVATION_EXTRACTION_PROMPT`, asking for observations across five categories: tool patterns, success factors, failure patterns, cost patterns, and time patterns. Observations with confidence < 0.5 are filtered. Survivors are written as `OBSERVATION` nodes under the Experience tree's Observations section, embedded via `EmbeddingService.embed_node`, with `importance_score = confidence`.

6.3 Phase 2 — Pattern Recognition

Observations are clustered by greedy embedding-cosine similarity (> 0.75). For each cluster of size ≥ 2 :

1. The LLM is called with `PATTERN_RECOGNITION_PROMPT` over the cluster's summaries.
2. A `PATTERN` node is written with `metadata_extra = {source_observations, pattern_strength, recurrence_count, success_correlation}`.
3. For each source observation, a `CortexEdge` (`edge_type = "derived_from", weight = 1/|cluster|, created_by = "dreaming_engine"`) is created from `pattern → observation`.

6.4 Phase 3 — Intelligence Distillation

Strong patterns (strength ≥ 0.7 , recurrence $\geq \text{MIN_PATTERNS_FOR_DISTILLATION} = 2$) are fed to the LLM with `INTELLIGENCE_DISTILLATION_PROMPT` alongside existing rule summaries (for de-duplication). Output is an array of typed rules: instruction, strategy, or preference. Each rule becomes an `INSTRUCTION`, `STRATEGY`, or `PREFERENCE` node under the appropriate Intelligence section. The tree's `consolidation_generation` is incremented and `last_consolidated_at` is set.

6.5 Properties of the Consolidation Pipeline

- **Monotonic accumulation.** Existing rules are not deleted; new generations add on top. Conflict resolution between generations is reserved for the `contradicts` and `supersedes` edge types (future work).
- **Confidence-weighted retrieval.** Rule retrieval at runtime (`IntelligenceTreeService.get_applicable_` ranks by `confidence × cosine_similarity`, then bulk-updates `access_count` and `last_accessed_at`. This implements a form of frequency-weighted importance.
- **Domain coupling via edges.** The `derived_from` and (future) `generalizes` edges create a back-pointer chain from `rule → pattern → observation → episode`, making every rule fully traceable to the runs that produced it.

7 The Semantic Graph Layer

The `cortex_edges` overlay provides associative cross-tree retrieval.

7.1 Hybrid Search

`SemanticGraphService.semantic_graph_search` proceeds in three steps:

1. Embed the query with `task_type = RETRIEVAL_QUERY`.
2. **Seed.** Run pgvector cosine search restricted to (`ct.entity_id = :entity_id` OR `ct.scope_level IN ('app', 'tenant')`) and optionally to specified `memory_domains`. Return top- k seeds.
3. **Expand.** For each seed, run BFS via a recursive CTE with cycle protection (`NOT (target_node_id = ANY(path))`), `depth ≤ expansion_depth`, traversing edges with `weight ≥ min_weight`. Multiply weights along paths.
4. **Re-rank.** Combine: `combined_score = 0.7 · similarity + 0.3 · edge_weight`. Sort descending, truncate to $2k$.

This couples the precision of embedding search with the recall of graph expansion.

7.2 Auto-Edge Creation

After any node is embedded, `create_similarity_edges(node_id)` finds up to 5 nodes with cosine similarity ≥ 0.85 and creates `semantic_similar` edges weighted by the similarity score.

7.3 Co-Access Tracking

`track_co_access(node_ids, run_id)` creates pairwise `co_accessed` edges (initial weight 0.3) between all nodes the agent reads in the same step. Subsequent re-traversals boost the weight by `BOOST_ON_TRAVERSAL = 0.05`, capped at 1.0. Over time the graph organizes along trajectories of nodes that are useful together.

7.4 Maintenance

A daily `graph_maintenance_worker`:

```
-- Decay weights of edges not traversed in 30 days
UPDATE cortex_edges
SET weight = GREATEST(MIN_WEIGHT, weight * 0.95)
WHERE last_traversed_at < NOW() - INTERVAL '30 days'
OR last_traversed_at IS NULL;

-- Prune edges below minimum
DELETE FROM cortex_edges WHERE weight < MIN_WEIGHT;
```

Decay plus prune produces a self-trimming graph whose size scales with active relevance rather than accumulated history.

8 Scope Hierarchy

CORTEX implements a six-level scope hierarchy via the `scope_level` enum, allowing the same schema to represent platform-wide, multi-tenant, per-user, per-agent, and per-run state (Table 4).

The `semantic_graph_search` SQL clause (`ct.entity_id = :entity_id` OR `ct.scope_level IN ('app', 'tenant')`) is the inheritance primitive: an entity sees its own trees plus shared app/tenant trees, without duplicating data.

9 Implementation

9.1 Stack

- **Language.** Python 3.10+.

Table 4: The six scope levels and their typical uses.

Level	Value	Inherits to	Typical use
L0	app	All tenants	Platform-wide instructions shared across all tenants
L1	partner	Partner’s tenants	Shared across tenants under a partner organization
L2	tenant	Tenant’s users	Tenant-level knowledge (company-wide playbook)
L3	user	User’s entities	Per-user preferences
L4	entity	Entity’s runs	Per-agent persistent trees
L5	runtime	—	One per execution (working memory)

- **Async ORM.** SQLAlchemy 2.x with asyncio support.
- **Database.** PostgreSQL with the `pgvector` extension.
- **Embeddings.** Pluggable; reference implementation uses Vertex AI (`text-embedding-005`, 768-dim) but the resolution path supports per-tenant override via an admin configuration table.
- **LLM.** Pluggable via an internal `LLMRouter` (reference implementation supports OpenAI, Anthropic, and Google providers).
- **Background jobs.** `arq` (Redis-backed) for `dreaming_worker`, `graph_maintenance_worker`, `cortex_resume_scheduled` (cron, every 5 minutes), `resume_execution`, and `process_document`.

9.2 Execution Engine Integration

The reference `ExecutionEngine` invokes CORTEX in five phases (C1–C5):

```

C1: Create or resume CORTEX tree
C2: Memory assembly (assemble_memory facade routes v1/v2)
C3: Build context from viewport, knowledge subtree, context sources
C4: Locate working memory root
C5: Execute plan steps with per-step write_step, refresh_viewport,
    and periodic write_checkpoint

```

CORTEX-native step types (`NAVIGATE`, `READ`, `WRITE`, `RECURSE`, `AWAIT_CHILDREN`) are handled by `CortexBridge.execute_cortex_step`. Conventional step types (`THOUGHT`, `ACTION`, `TOOL_CALL`, `CHILD_ENTITY_INVOCATION`) are handled by the step executor, which post-processes by writing each step result as a `finding` node and, after tool calls, ingesting tool output as `knowledge` nodes with provenance metadata.

9.3 Performance Engineering

Several optimizations were applied during development:

- **Recursive CTEs replace iterative parent walks.** `_build_breadcrumb` and `_is_descendant_of` use single recursive CTE queries, replacing $O(\text{depth})$ sequential `SELECT`s with one round-trip.
- **Incremental context-size tracking.** `update_context_size(key, old, new)` mutates an $O(1)$ counter on every `store_step_output`, replacing $O(n)$ full-context scans before auto-compaction checks.
- **Redis viewport caching.** `refresh_viewport` caches the rendered viewport text under `cortex:viewport:{tree.id}:{cursor_id}` with a 30 s TTL, avoiding redundant tree traversals when the cursor has not moved between steps.

- **Batch embedding.** `EmbeddingService.embed_batch` batches at `BATCH_SIZE = 100` (the Vertex AI limit) with per-text truncation to 8000 characters.
- **UUID caching for long sessions.** `working_root_id`, `_tree_output_root_id`, and `_tree_id` are captured as locals before the first session commit to avoid `MissingGreenlet` errors from ORM-attribute expiry during long step loops.

9.4 Safety and Multi-Tenancy

- **Tenant isolation.** Every `CortexRouter` is constructed with a `company_id`; all SQL filters by it.
- **Subtree isolation.** See §5.4.
- **Sensitive-context redaction.** A `_SENSITIVE_CONTEXT_KEYS` set redacts substrings like `api_key`, `secret`, `token`, `password` from `context_state` before any database write.
- **Parent → child memory leak prevention.** When the execution engine spawns a child via `CHILD_ENTITY_INVOCATION`, it strips parent-scoped memory keys (`__memory__`, `__episodic_memory__`, `__semantic_context__`, etc.) before child invocation.
- **Null-byte sanitization.** Auto-ingested context sources are stripped of `\x00` bytes that PostgreSQL UTF-8 columns reject.
- **REST authorization.** All `/api/v1/cortex/*` endpoints require authentication; `company_id` is taken from the authenticated user, never from the request body.

10 Discussion: Design Properties and Trade-Offs

10.1 Properties

Table 5 summarizes the system’s design properties and the mechanisms that produce them.

Table 5: Design properties and the mechanisms that produce them.

Property	Mechanism
Bounded per-step prompt cost	Viewport contains at most $M + \text{depth}(n)$ summaries
Cross-run continuity	Four persistent entity-scoped trees
Deterministic resumability	<code>resume_cursor_id</code> + checkpoint nodes + <code>__completed_steps__</code>
Sub-agent isolation	<code>scoped_subtree_root_id</code> with recursive-CTE ancestry check
Traceable provenance	<code>source_ref</code> + <code>metadata_extra.provenance_chain</code> per node
Self-organizing graph	Edge weight boost on traversal, decay on inactivity, prune on threshold
Domain extensibility	Adding a domain or scope level is an enum extension, not a schema change

10.2 Trade-Offs

- **Postgres-only.** The recursive CTEs and pgvector dependencies preclude SQLite or MongoDB backends. This is by design — the structural query patterns are the architectural moat.
- **Async-only API.** Practical for modern Python agents but excludes synchronous codebases.
- **Operational complexity.** Running CORTEX in production requires PostgreSQL, pgvector, Redis, and an arq worker. This is higher operational overhead than a library like Mem0 that runs in-process.

- **LLM cost during consolidation.** Dreaming makes three LLM calls per cycle. At typical observation/pattern volumes, this is sub-dollar per entity per day but is not free.
- **Greedy clustering in dreaming.** `_cluster_observations` is $O(n^2)$. Degrades past a few hundred observations; a HNSW-backed clustering pass is planned (§12).
- **Synchronous re-clustering.** Triggered inside `write()` rather than queued; can briefly block step execution under high write pressure.
- **No node-level ACLs.** The unit of access is the tree. Per-node ACLs are reserved for future work.

10.3 When Cortex Is and Is Not the Right Choice

CORTEX is well-suited to:

- Long-horizon agents (multi-step, multi-hour, multi-day tasks).
- Agents that must produce structured output documents.
- Multi-tenant SaaS platforms with per-customer learning loops.
- Multi-agent systems where sub-agent isolation is critical.

CORTEX is overkill for:

- Single-turn chatbots needing only conversation history.
- Stateless RAG over a fixed document corpus (PageIndex or vanilla vector RAG suffice).
- Agents whose memory needs are exhausted by Mem0-style fact extraction.

11 Limitations

We list limitations honestly to delineate the system’s current scope:

1. **No empirical benchmarks yet.** This paper is a system description. Standard agent-memory benchmarks (LoCoMo, LongMemEval, AgentBench) are in the planning phase. No numerical claims about retrieval accuracy, downstream task completion, or token cost reduction are made here.
2. **Dreaming has no automatic post-run trigger.** `dreaming_worker` must be explicitly enqueued; a hook chained from `MemoryRouter.write_episodic` is planned.
3. **Memory scope `RUN_SCOPED` is currently identical to `FULL`.** The filter to restrict retrieval to the current run’s tree is not yet implemented.
4. **Tree status remains active after run completion.** Long-term archival sweeps are not yet designed.
5. **Concurrent writes under the same parent** rely on `MAX(sibling_order) + 1` without `SELECT FOR UPDATE`; the race window is small but not formally prevented.
6. **v1 and v2 retrieval pipelines coexist.** The `MemoryAssemblyService` (v2) is implemented but the default per-entity pipeline is v1. Migration is gated by a feature flag on the entity capability config.

7. **Reserved edge types** (`generalizes`, `contradicts`, `supersedes`, `applies_to`) are defined in the vocabulary but not yet emitted by the system.
8. **No formal quality gate on distilled rules.** Low-confidence LLM outputs can pollute the Intelligence tree; a validation pass is planned.

12 Future Work

12.1 Empirical Evaluation

Planned experiments:

- **Long-horizon task coherence** (analogous to Letta’s 30-day continuous-run benchmark).
- **Cross-session fact recall** (LoCoMo / LongMemEval comparison vs Mem0, Zep, Letta).
- **Cost per task** (Dreaming amortization vs naive context-stuffing baselines).
- **Sub-agent isolation correctness** (formal verification of Property 2 via fuzz testing).
- **Retrieval ablations:** pure embedding vs embedding + graph-1-hop vs embedding + graph-2-hop.

12.2 Algorithmic Improvements

- Replace greedy $O(n^2)$ observation clustering with HNSW-backed batched clustering.
- Implement automatic post-run dreaming triggers gated by run count and time since last consolidation.
- Add LLM-based quality gate before writing Intelligence rules.
- Implement `contradicts`/`supersedes` edge emission during distillation; resolve conflicts at retrieval time.

12.3 Operational Improvements

- Node-level ACLs for fine-grained sharing.
- Tree archival sweeps (move `complete` and inactive $> N$ days trees to a cold store).
- Async (queued) re-clustering instead of synchronous in `write()`.
- Backend-agnostic abstraction (although the design will remain Postgres-first).

12.4 Theoretical Work

- Formalize the relationship between viewport bounded-context and downstream task accuracy.
- Connect the Dreaming consolidation pipeline to formal models of memory consolidation in cognitive science (e.g., the two-stage hippocampal-neocortical model).
- Investigate whether Intelligence rules form a useful “policy distillation” target for fine-tuning smaller models.

12.5 Ecosystem

Open-source as a standalone pip-installable package, with backend plugins for OpenAI / Anthropic / litellm LLM providers and Vertex / OpenAI / sentence-transformers embedding providers (planned).

13 Conclusion

We presented CORTEX, an agent-memory architecture in which the agent’s complete cognitive state is a typed, persistent, navigable, writable tree, and the language model’s context window is a bounded viewport onto that tree. CORTEX unifies four memory domains and a transient runtime workspace on a single ORM model trio, with seven primitive operations, a three-phase background consolidation pipeline, a semantic graph overlay, and a six-level scope hierarchy. Each individual element draws on prior work — MemGPT’s tiering, PageIndex’s hierarchical navigation, RLM’s recursive decomposition, Park et al.’s reflection trees, CoALA’s memory taxonomy — but the integration into one coherent runtime with one schema and one set of primitives is, to our knowledge, novel. We have characterized the system’s design properties, documented its limitations, and outlined a path to empirical evaluation. The architecture is in production use; the source is available for inspection and extension.

Acknowledgments

We thank the broader community working on LLM agent memory — including the teams behind MemGPT/Letta, Mem0, Zep, Cognee, Generative Agents, PageIndex, and the CoALA framework — for the foundational ideas this work builds upon. Any errors or unsupported claims remain our own.

References

- [1] C. Packer, S. Wooders, K. Lin, et al. MemGPT: Towards LLMs as Operating Systems. *arXiv preprint arXiv:2310.08560*, 2023.
- [2] P. Bansal et al. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. *arXiv preprint arXiv:2504.19413*, 2025.
- [3] J. S. Park, J. C. O’Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein. Generative Agents: Interactive Simulacra of Human Behavior. In *Proceedings of UIST*, 2023.
- [4] T. R. Sumers, S. Yao, K. Narasimhan, and T. L. Griffiths. Cognitive Architectures for Language Agents. *arXiv preprint arXiv:2309.02427*, 2023.
- [5] J. E. Laird. Introduction to the Soar Cognitive Architecture. *arXiv preprint arXiv:2205.03854*, 2022.
- [6] Vectify AI. PageIndex: Document Index for Vectorless, Reasoning-Based RAG. Technical report and open-source release, 2025. <https://github.com/VectifyAI/PageIndex>.
- [7] A. L. Zhang. Recursive Language Models. *arXiv preprint arXiv:2512.24601*, 2025.
- [8] J. Wang et al. G-Memory: Tracing Hierarchical Memory for Multi-Agent Systems. In *Advances in Neural Information Processing Systems*, 2025.
- [9] ByteRover Team. ByteRover: Agent-Native Memory Through LLM-Curated Hierarchical Context. *arXiv preprint arXiv:2604.01599*, 2026.

- [10] S. Liu et al. Cognitive Workspace: Active Memory Management for LLMs — An Empirical Study of Functional Infinite Context. *arXiv preprint arXiv:2508.13171*, 2025.
- [11] Letta Inc. Benchmarking AI Agent Memory: Is a Filesystem All You Need? Letta blog, 2025. <https://www.letta.com/blog/benchmarking-ai-agent-memory>.
- [12] Anonymous. Rethinking Memory Mechanisms of Foundation Agents in the Second Half: A Survey. *arXiv preprint arXiv:2602.06052*, 2026.

A Glossary

Term	Definition
CORTEX	Cognitive Orchestrated Recursive Tree EXecution
Viewport	The bounded representation of the agent’s current tree position
Cursor	<code>tree.resume_cursor_id</code> — the node the agent is “at”
Domain	One of {knowledge, episodic, experience, intelligence}
Scope	Six-level hierarchy {app, partner, tenant, user, entity, runtime}
Dreaming	The three-phase background consolidation pipeline
Subtree anchor	A direct child of the runtime tree’s root (Knowledge / Working / Output)
Working memory	The subtree where step <code>finding</code> nodes accumulate
Episode	A row in the persistent Episodic Tree representing one completed run
Observation	An LLM-extracted insight about a cluster of episodes
Pattern	A recurring behavior synthesized from clustered observations
Rule	An actionable directive distilled from strong patterns

B The Seven Operations — Pseudocode Reference

```
# 1. NAVIGATE -- bounded read of one level of the tree
async def navigate(node_id) -> Viewport:
    node = await _get_node(node_id)
    tree.resume_cursor_id = node_id
    children = SELECT * FROM cortex_nodes
                WHERE parent_id = node_id ORDER BY sibling_order
    parent = await _get_node(node.parent_id) if node.parent_id else None
    breadcrumb = RECURSIVE_CTE_walk_to_root(node_id)
    return Viewport(node, children, parent, breadcrumb)

# 2. READ -- paged content access
async def read(node_id, page=0) -> NodeContent:
    node = await _get_node(node_id)
    if node.status == PENDING: node.status = ACTIVE
    tree.resume_cursor_id = node_id
    page_chars = tree.page_size_tokens * CHARS_PER_TOKEN
    return node.content[page * page_chars : (page + 1) * page_chars]

# 3. WRITE -- create child node; enforce invariants
async def write(parent_id, node_type, title, content, summary, ...) -> UUID:
    parent = await _get_node(parent_id)
    assert parent.summary, "Invariant 1: parent must have summary"
    if child_count(parent_id) >= tree.max_children:
        await _schedule_reclustering(parent_id)
    sibling_order = SELECT COALESCE(MAX(sibling_order), -1) + 1
                    FROM cortex_nodes WHERE parent_id = parent_id
    INSERT new node
    tree.total_nodes += 1
    tree.resume_cursor_id = new_node.id
    return new_node.id
```



```

# 4. RECURSE -- scoped subtree execution
async def recurse(node_id, task, result_slot) -> (UUID, UUID):
    task_node_id = await write(
        parent_id=node_id, node_type="task",
        content={task, result_slot, scoped_to: node_id}, ...
    )
    child_run = ExecutionRun(
        parent=current,
        input={cortex_tree_id, subtree_root_id: node_id, task, ...}
    )
    enqueue_async("execute_run", child_run.id)
    return (task_node_id, child_run.id)

# 5. AWAIT_CHILDREN -- collect completed child tasks
async def await_children(parent_node_id) -> Dict[str, NodeSummary]:
    children = SELECT * FROM cortex_nodes
        WHERE parent_id = parent_node_id
        AND node_type = 'task' AND status = 'complete'
    return {c.metadata.result_slot: NodeSummary(c) for c in children}

# 6. CHECKPOINT -- compress context into a node
async def checkpoint(tree_id, progress, key_facts, next_steps) -> UUID:
    cursor = tree.resume_cursor_id
    return await write(
        parent_id=cursor, node_type="checkpoint",
        content={progress, key_facts, next_steps,
            nodes_written, time_elapsed}, ...
    )

# 7. ASSEMBLE -- DFS output subtree + bridge paragraphs
async def assemble_output(tree_id, coherence_pass=True) -> str:
    sections = DFS_collect(tree.output_root_id, where status='complete')
    if coherence_pass and len(sections) > 1:
        bridges = LLM(
            prompt="Write transitions between sections", sections
        )
        return interleave(sections, bridges)
    return "\n\n".join(sections)

```

C Reproducibility

A reference implementation is available at the project repository (URL to be released with the package) under an OSI-approved license. Postgres schema migrations are shipped with the package. To reproduce the system locally:

```

# Prerequisites: PostgreSQL 14+ with pgvector, Redis
pip install cortex-memory
createdb cortex_dev && psql cortex_dev -c "CREATE EXTENSION vector;"
cortex-memory migrate --database-url postgresql+asyncpg://localhost/cortex_dev

# Minimal usage
from cortex_memory import CortexRouter, OpenAIEmbeddings, OpenAILLM
router = CortexRouter(
    db=session, tenant_id=..., embeddings=..., llm=...
)
tree = await router.create_tree(owner_id=..., task="Draft Q3 report")
node_id = await router.write(
    parent_id=tree.root_node_id, node_type="finding",
    title=..., summary=..., content=...
)

```